Reporte Técnico de Actividades Práctico-Experimentales Nro. 001

1.Datos Generales

Nombre del estudiante(s)	Darwin Enrique Granda Orosco Darwin Andrés Sarango Soto Joseph Sebastian Balcazar Loaiza
Asignatura	Software Security
Ciclo	8 A
Unidad	1
Resultado de Aprendizaje de la Unidad	R1. Diseña e implementa software bajo los principios básicos de estándares seguros, en el marco de los principios de solidaridad, transparencia, responsabilidad y honestidad.
Práctica Nro.	1 (Fases 1 a 4)
Título	Análisis y superficie de ataque del proyecto
Docente	Valeria Herrera Salazar
Período	Del 28 de septiembre al 23 de octubre de 2026 (Semanas 1 a 4)
Horario	Jueves: 09h30 – 11h30 Viernes: 07h30 – 09h30
Lugar	Aula de Clase
Tiempo planificado	16 horas (4 horas por semana)

(Abarca APE 1, 2, 3 y 4 - Semanas 1 a 4)

2. Objetivo(s) de la Práctica:

La primera guía inicia el proyecto integrador que acompañará a toda la asignatura: cada estudiante o pareja de trabajo selecciona la aplicación que auditará durante las dieciséis semanas y construye, sobre ella, la línea base de seguridad del proyecto. Cada fase responde de manera directa al contenido teórico tratado en las horas de contacto con el docente de esa misma semana.

- Seleccionar la aplicación que se auditará durante las dieciséis semanas del ciclo (un proyecto propio de asignaturas anteriores o un entorno controlado como DVWA u OWASP Juice Shop).
- Delimitar la superficie de ataque de la aplicación y elaborar el inventario inicial de elementos expuestos.
- Aplicar la tríada CIA (confidencialidad, integridad y disponibilidad) para clasificar los activos y las amenazas del proyecto.
- Revisar la gestión de errores y excepciones, detectando las fugas de información en mensajes y trazas de la aplicación.
- Evaluar la aplicación frente al OWASP Top 10 y elaborar la matriz de priorización de hallazgos que servirá de insumo directo para la auditoría técnica de la guía APE 02.

3. Materiales y reactivos:

Los recursos necesarios para esta guía están disponibles en la infraestructura institucional y en herramientas de libre acceso. El docente publicará en el EVA, con anticipación a cada semana, las plantillas de registro correspondientes a cada fase.

- Ordenador personal con acceso a internet y navegador web actualizado.
- Cuenta activa en GitHub u otra plataforma similar para el repositorio de evidencias del proyecto (una cuenta por pareja o por estudiante).
- Aplicación objetivo: proyecto propio de asignaturas previas (con código disponible, ejecución local, autenticación y base de datos) o entorno controlado (DVWA u OWASP Juice Shop) instalado en una máquina virtual.
- Plantillas del EVA: ficha del proyecto, inventario de superficie de ataque, matriz de activos y amenazas, registro de gestión de errores y lista de cotejo OWASP Top 10.
- Material de consulta: la entrada oficial del OWASP Top 10 y los documentos de clase de las semanas 1 a 4.

4. Equipos y herramientas:

- Aula física con equipamiento audiovisual.
- Computadoras personales o dispositivos móviles de los estudiantes.
- Entorno Virtual de Aprendizaje (EVA) de la UNL.
- Repositorio GitHub del proyecto para el portafolio de evidencias.
- Software de hojas de cálculo (Excel/Google Sheets) para la matriz de activos y amenazas y para la matriz de priorización de hallazgos.
- Procesador de texto para la redacción de los informes de cada fase.

5. Resultados esperados:

FASE 1. Selección, alcance y superficie de ataque

- **Repositorio del proyecto en GitHub constituido, con la ficha del proyecto y el inventario de superficie de ataque con evidencias.**

  Link al repositorio de aplicación: https://github.com/DarwinG23/Server5K

  Link al repositorio Registro del proyecto : https://github.com/josephsebal/Server5K-Seguridad.git

  Ficha del proyecto

Campo	Descripción
Nombre	Server5K – Sistema de competencias 5K
Función	Gestionar competencias, jueces, equipos y registros de tiempo en milisegundos, con actualización en tiempo real.
Usuarios	Jueces autenticados, operador/administrador y visitantes de la vista pública.
Roles	Juez: consulta sus equipos y registra tiempos. Administrador: gestiona el sistema mediante Django Admin. Visitante: consulta resultados públicos.
Backend	Django 6.0, Django REST Framework, Daphne/ASGI.

Tiempo real	Django Channels, WebSocket y Redis.
Persistencia	PostgreSQL 16 en Docker; SQLite como alternativa local.
Autenticación	JWT mediante SimpleJWT; autenticación propia para el modelo Juez.
Documentación	OpenAPI/Swagger en /api/docs/ y ReDoc en /api/redoc/.
Despliegue	Docker Compose con servicios web, PostgreSQL y Redis.

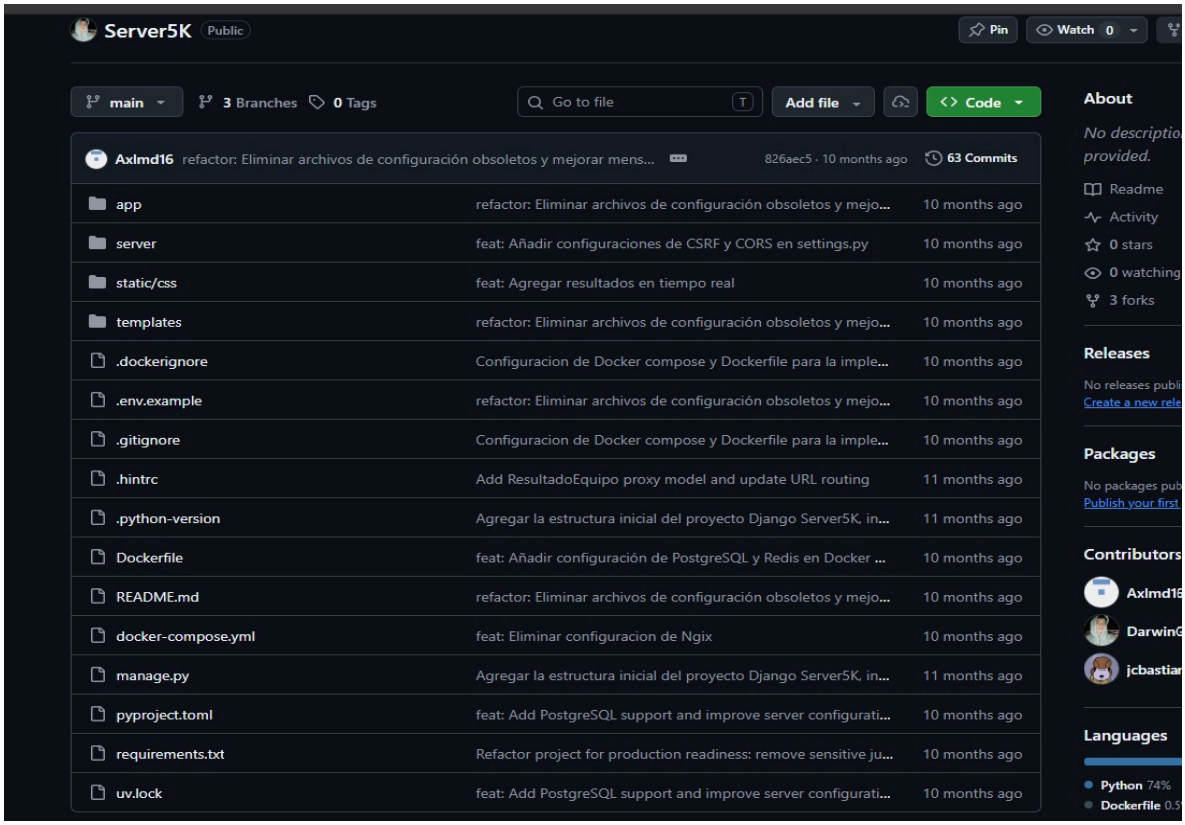


Fig 1. Repositorio de aplicación Server5K

Criterios de selección de la aplicación

Requisito de la guía	Verificación inicial
Código disponible	Cumple: código Python/Django presente en el repositorio.
Ejecución local	Preparada: Docker Compose documentado. Falta adjuntar captura de ejecución a esta ficha.
Autenticación	Cumple: dispone de login, JWT, refresh, logout y /api/me/.
Base de datos	Cumple: PostgreSQL configurado y SQLite para desarrollo.

Entorno autorizado	Cumple si todas las pruebas se ejecutan localmente y dentro del entorno autorizado.

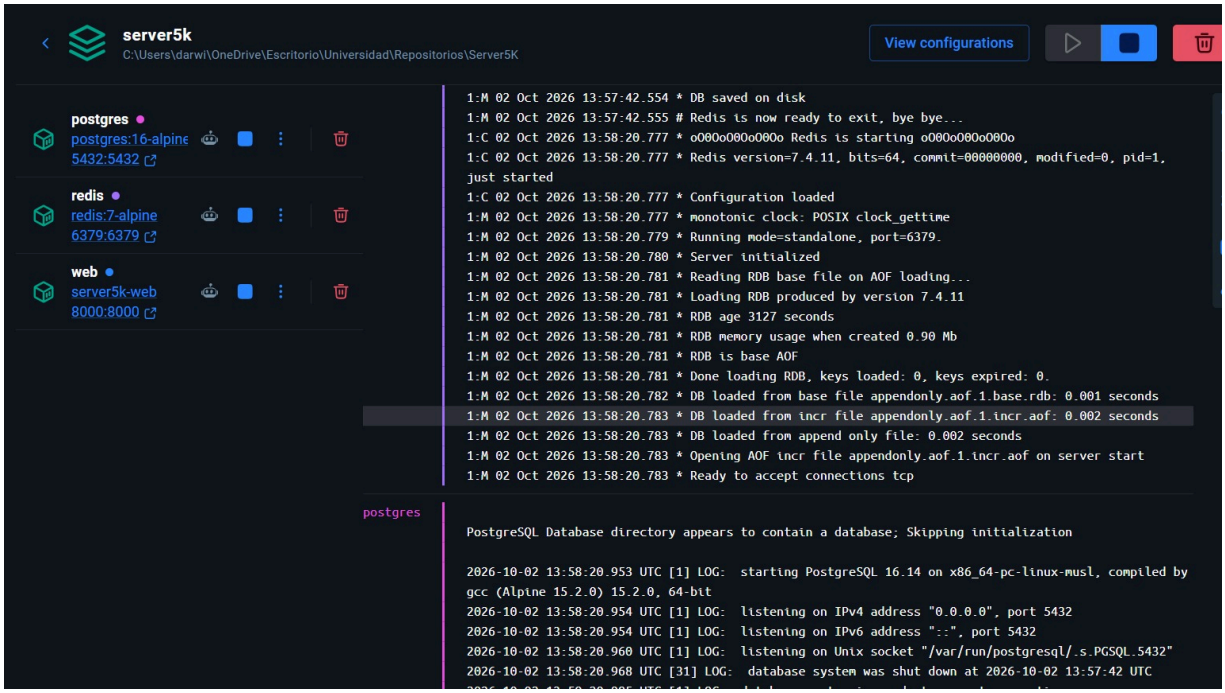


Fig 2. Servicios y registros de Docker Compose (Aplicación funcionando)

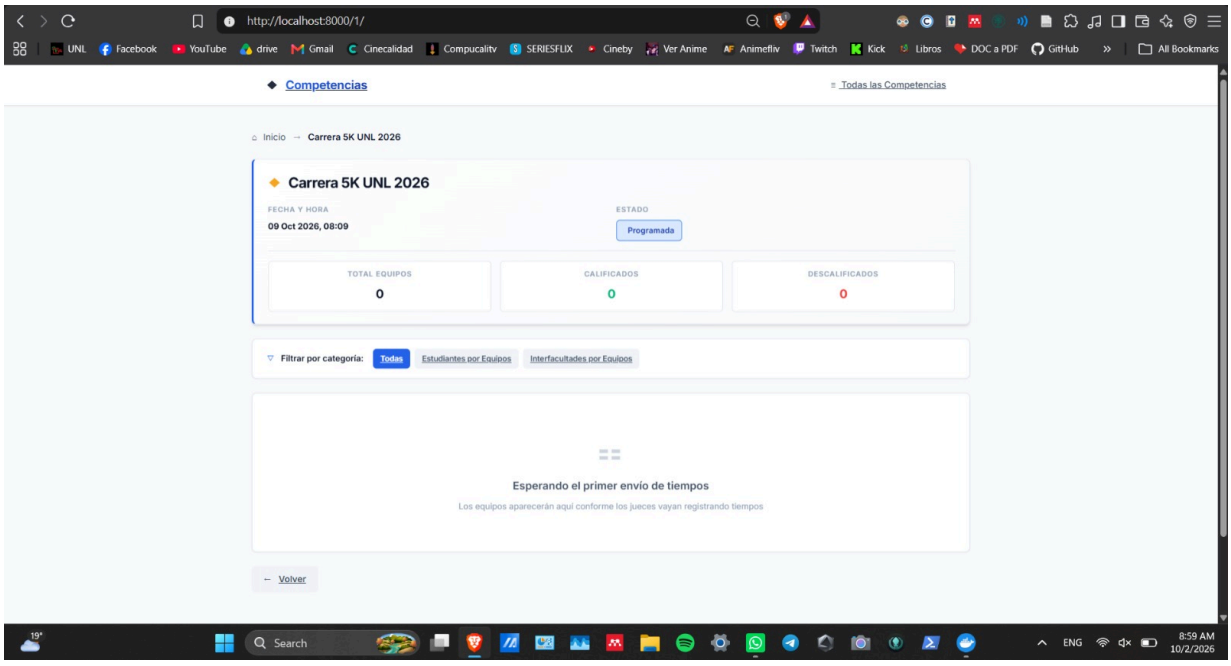


Fig 3. Interfaz pública de competencias

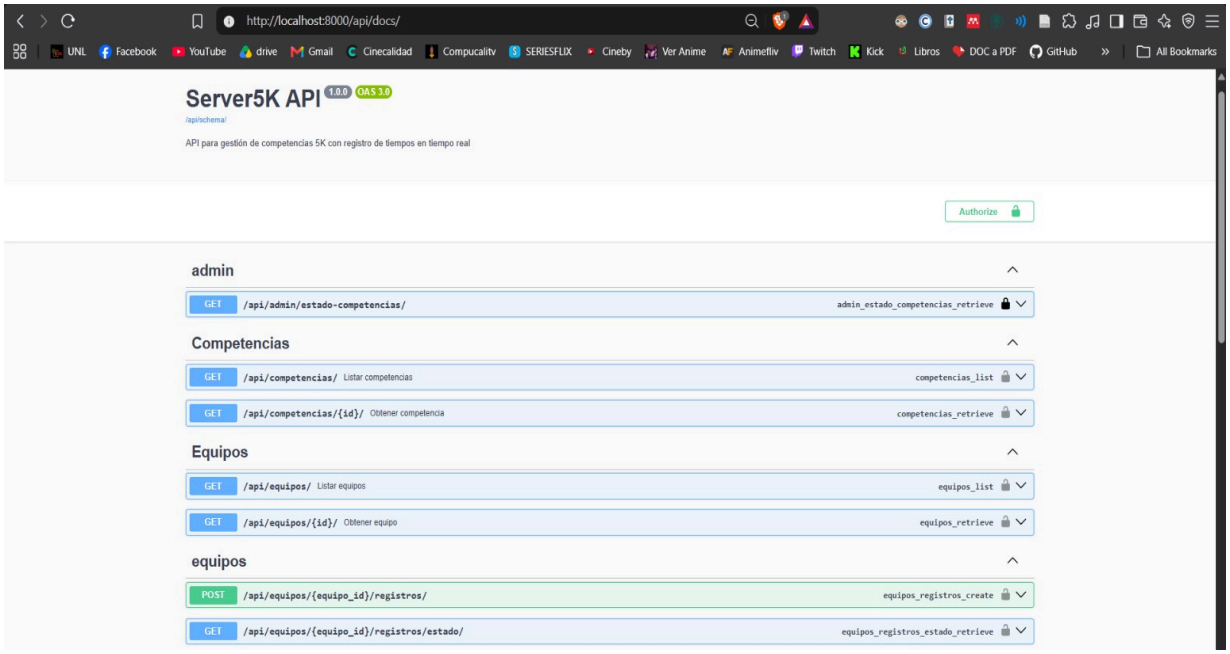


Fig 4. Documentación de API REST.

Alcance de la auditoría

Tipo	Componentes y funcionalidades
Dentro del alcance	Autenticación y sesiones JWT, Django Admin, API REST, control de acceso por juez y equipo, registro de tiempos, WebSocket, vistas públicas, configuración de seguridad, gestión de errores, cabeceras HTTP, CORS/CSRF, secretos y dependencias.
Fuera del alcance	Infraestructura del proveedor de hosting, dispositivos físicos de cronometraje, disponibilidad real de Internet, seguridad física y datos reales de competidores.

Inventario de superficie de ataque

Nro	Elemento expuesto	Ubicación	Actor
SA-01	Login personalizado	POST /api/login/	Cualquier cliente
SA-02	Refresh personalizado	POST /api/token/refresh/	Cliente con refresh token
SA-03	Refresh estándar duplicado	POST /api/token/refresh/ en server/urls.py	Cliente con refresh token
SA-04	Logout y blacklist	POST /api/logout/	Juez autenticado

SA-05	Perfil propio	GET /api/me/	Juez autenticado
SA-06	Competencias	GET /api/competencias/, /api/competencias/<id>/	Juez autenticado
SA-07	Equipos	GET /api/equipos/, /api/equipos/<id>/	Juez autenticado
SA-08	Registro masivo de tiempos	POST /api/equipos/<id>/registros/	Juez autenticado
SA-09	Estado de registros	GET /api/equipos/<id>/registros/estado/	Juez autenticado
SA-10	Estado público de competencias	GET /api/admin/estado-competencias/	Anónimo
SA-11	Documentación OpenAPI	/api/schema/, /api/docs/, /api/redoc/	Anónimo, según configuración
SA-12	Vista pública	/, /<id>/, /equipo/<id>/	Anónimo
SA-13	WebSocket de juez	ws://.../ws/juez/<id>/?token=...	Juez con JWT
SA-14	Panel administrativo Django	/admin/	Administrador
SA-15	Archivos multimedia y estáticos	/media/, /static/	Cliente web
SA-16	Health check	GET /api/health/	Anónimo

FASE 2. Tríada CIA, activos, amenazas y controles

- **Matriz de activos y amenazas clasificada según la tríada CIA, con controles propuestos para los activos críticos.**

  A partir de la arquitectura identificada se consideraron activos de información, credenciales, servicios de comunicación y mecanismos de registro. Cada activo se valoró en confidencialidad (C), integridad (I) y disponibilidad (A) con la escala cualitativa Alta (A), Media (M) o Baja (B).

Tabla 1. Criterios de valoración de la tríada CIA

Criterio	Alta (A)	Media (M)	Baja (B)
Confidencialidad	Divulgación de datos sensibles, credenciales o acceso privilegiado.	Exposición de información operativa interna.	Información destinada a consulta pública.
Integridad	La manipulación altera resultados, roles o decisiones críticas.	Los cambios afectan funciones recuperables del sistema.	Modificaciones con efectos menores y fáciles de revertir.
Disponibilidad	La interrupción impide realizar actividades esenciales del sistema.	La interrupción afecta funciones secundarias sin detener completamente el servicio.	La indisponibilidad genera un impacto operativo mínimo.

Tabla 2. Identificación y clasificación de activos según la tríada CIA

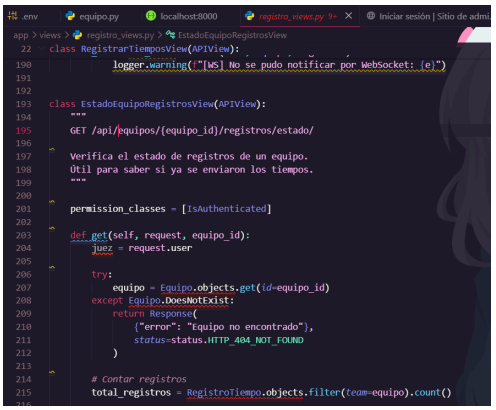
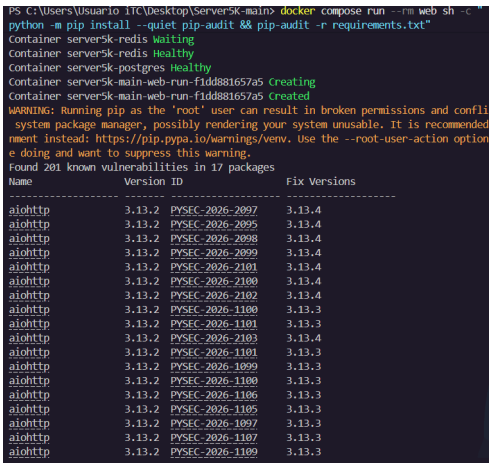
Activo	C	I	D	Amenaza	Controles actuales	Prioridad	Controles propuestos
Credenciales de jueces	Alto	Alto	Medio	Robo, fuerza bruta o reutilización	Hash de contraseña, JWT	Alta	1. Limitar los intentos fallidos de inicio de sesión para evitar ataques de fuerza bruta. 2. Implementar la verificación en dos pasos y establecer contraseñas seguras al crear cuentas.
Claves JWT y refresh tokens	Alto	Alto	Alto	Robo, falsificación o replay	Firma HS256, blacklist	Alta	1. Renovar periódicamente los tokens y permitir su revocación cuando sea necesario. 2. Proteger los tokens mediante conexiones HTTPS y cookies seguras.
Datos personales de jueces	Alto	Medio	Medio	Enumeración o acceso de otro juez	/me/ limitado al usuario autenticado	Alta	1. Mostrar únicamente los datos personales necesarios para cada función. 2. Verificar que cada juez solo pueda acceder a su información y registrar los accesos realizados.

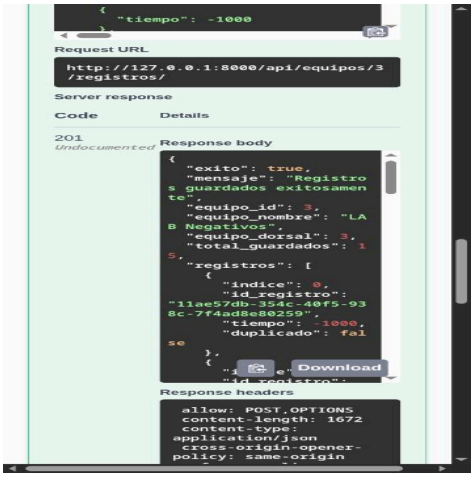
Asignación juez-equipo	Medio	Alto	Alto	Manipulación de IDs/IDOR	Filtro por juez y validación en POST	Alta	1. Comprobar que cada juez tenga permiso para consultar o modificar sus asignaciones. 2. Realizar pruebas para evitar que un juez acceda a las asignaciones de otro.
Datos de competencia	Bajo	Alto	Alto	Alteración de estado o borrado	Admin y relaciones Django	Alta	1. Establecer permisos según el rol de cada usuario para evitar modificaciones no autorizadas. 2. Registrar los cambios realizados y crear copias de seguridad que puedan recuperarse.
Registros de tiempo	Medio	Alto	Alto	Inyección de tiempos, duplicación o borrado	Transacción, UUID, idempotencia, validación	Alta	1. Validar que los tiempos registrados sean correctos y guardar un historial de los cambios realizados. 2. Realizar copias de seguridad y comprobar que los registros puedan recuperarse ante posibles fallos.
Resultados públicos	Bajo	Alto	Alto	Manipulación o indisponibilidad durante evento	Vistas de solo lectura	Media	1. Utilizar un sistema de caché que permita consultar los resultados rápidamente sin alterar la información original. 2. Supervisar el funcionamiento del sistema y generar alertas ante errores o interrupciones.
Mensajes y canales WebSocket	Medio	Alto	Alto	Suplantación, escucha o saturación	JWT y verificación de juez_id	Alta	1. Proteger la comunicación mediante conexiones seguras (WSS) y tokens con tiempo de validez limitado. 2. Establecer límites en el tamaño y la cantidad de mensajes para evitar la saturación del sistema.
Base de datos	Alto	Alto	Alto	Exfiltración, corrupción o ransomware	PostgreSQL /SQLite según	Alta	1. Cifrar la información almacenada y permitir el acceso únicamente a

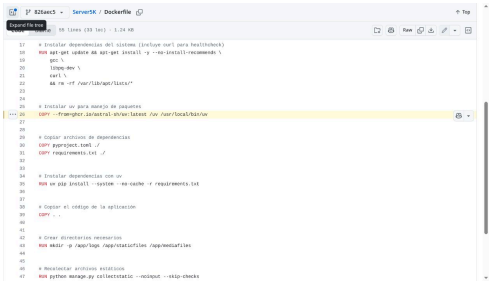
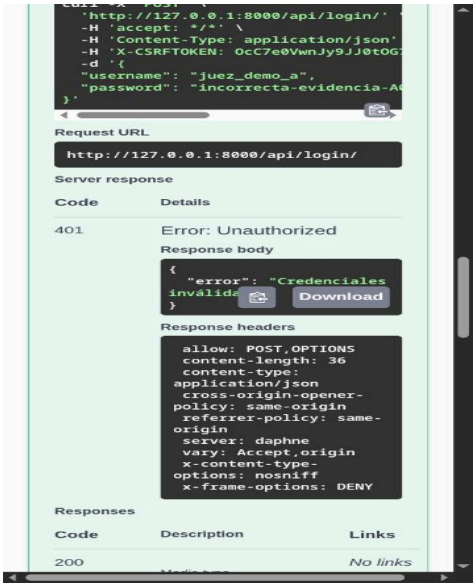
					entorno		usuarios y servicios autorizados. 2. Crear copias de seguridad cifradas y comprobar periódicamente que se puedan restaurar.
Logs y trazas	Alto	Medio	Medio	Tokens, rutas o datos sensibles en logs	Logging configurado ; no registra querystring WS	Alta	1. Evitar que se almacenen contraseñas, tokens u otros datos sensibles en los registros del sistema. 2. Restringir el acceso a los registros y establecer cuánto tiempo deben conservarse.
Configuración y secretos	Alto	Alto	Alto	Clave por defecto, .env filtrado	Variables de entorno documentadas	Alta	1. Impedir que el sistema se inicie en producción si no se ha configurado una clave de seguridad válida. 2. Guardar las contraseñas y claves del sistema en un servicio seguro para evitar su exposición.
Código y dependencias	Medio	Alto	Medio	CVE o componente vulnerable	uv.lock/requirements.txt fijados	Media	1. Utilizar herramientas automáticas que detecten vulnerabilidades en las librerías utilizadas por el proyecto. 2. Revisar y actualizar periódicamente las dependencias para corregir problemas de seguridad conocidos.

FASE 3 y 4. Gestión de errores y fugas de información; Mapeo OWASP Top 10 y priorización de hallazgos

- **Informe de gestión de errores con las fugas documentadas, su severidad clasificada y las correcciones propuestas.**
- **Lista de cotejo OWASP Top 10 completada con evidencias y matriz de priorización de los hallazgos del proyecto.**

Categoría	Estado	Hallazgo	Evidencia / forma segura de comprobarlo	Captura
A01: Broken Access Control	Confirmado	El endpoint de estado de registros verifica que el usuario esté autenticado, pero no comprueba que el equipo le pertenezca.	En registro views.py, un juez autenticado podría consultar /api/equipos/{id}/registros/estado/ usando el ID de otro equipo. Debe responder 403, pero actualmente puede responder 200.	
A02: Security Misconfiguration	Confirmado	DEBUG=True, HTTPS desactivado y existe una clave secreta de desarrollo.	En .env y settings.py. python manage.py check --deploy detectó 19 advertencias, incluyendo DEBUG activo, SECRET_KEY débil, HTTPS desactivado y cookies inseguras.	Captura omitida en la copia publica para evitar divulgar claves, tokens o detalles de depuracion. Consultar las evidencias seguras del repositorio.
A03: Software Supply Chain Failures	Confirmado	Las dependencias están definidas, pero las imágenes python:3.13-slim y uv:latest no están fijadas a una versión exacta.	Se ejecutó pip-audit sobre las dependencias definidas en requirements.txt. El análisis identificó 201 vulnerabilidades conocidas en 17 paquetes. Entre los componentes afectados se encuentran Django, aiohttp, cryptography, Daphne, PyJWT y Django REST Framework. Se recomienda actualizar las dependencias a las versiones indicadas por la herramienta y volver a ejecutar el análisis después de realizar pruebas de compatibilidad.	

A04: Cryptographic Failures	Confirmado	La aplicación puede usar una SECRET_KEY débil y los WebSockets reciben el JWT por query string.	La clave aparece en .env. En consumers.py, el token se envía como ?token=..., lo que podría quedar registrado en historiales, proxies o herramientas de monitoreo.	Captura omitida en la copia publica para evitar divulgar claves, tokens o detalles de depuracion. Consultar las evidencias seguras del repositorio.
A05: Injection	No se encontró	El código utiliza el ORM de Django y no se observaron consultas SQL concatenadas, eval() o exec().	El intento SQL recibió 401 y el marcador HTML quedó escapado, los controles funcionaron en esos flujos.	Captura omitida en la copia publica para evitar divulgar claves, tokens o detalles de depuracion. Consultar las evidencias seguras del repositorio.
A06: Insecure Design	Confirmado	La aplicación permite guardar tiempos negativos, aunque no son válidos para una carrera.	En Swagger enviamos 15 tiempos de -1000 a POST /api/equipos/3/registros/. Respondió 201 e indicó que guardó los 15.	
A07: Authentication Failures	Confirmado	Las contraseñas de desarrollo siguen patrones simples como juez1, juez1123 y juez2123. Los tokens duran hasta 60 minutos y los refresh tokens hasta 7 días.	La generación está en populate_data.py y la configuración JWT en settings.py.	Captura omitida en la copia publica para evitar divulgar claves, tokens o detalles de depuracion. Consultar las evidencias seguras del repositorio.

A08: Software or Data Integrity Failures	No confirmado	No se demostró que Server5K aceptara software o datos alterados sin verificarlos. Se observó uv:latest en el Dockerfile como mejora de construcción2	Abrimos el archivo Dockerfile y vimos que la herramienta uv puede cambiar de versión cuando se vuelve a preparar la aplicación. No probamos cambiar esa herramienta por una alterada, por lo que no encontramos un fallo confirmado de A08.	
A09: Security Logging and Alerting Failures	Observación de mejora	Cuando falla el login, el log registra el rechazo, pero no identifica la cuenta intentada ni su origen.	En Swagger enviamos una contraseña incorrecta a POST /api/login/: respondió 401. Después revisamos el log, que guardó hora y ruta, pero no cuenta ni IP.	
A10: Mishandling of Exceptional Conditions	Confirmado	Algunas respuestas devuelven directamente el texto de excepciones internas al cliente.	En registro_views.py se devuelve str(e) en errores 500. Se debe enviar un mensaje genérico y guardar el detalle únicamente en logs.	Captura omitida en la copia publica para evitar divulgar claves, tokens o detalles de depuracion. Consultar las evidencias seguras del repositorio.

Priorización por impacto y probabilidad

Hallazgo	Categoría	Impacto	Probabilidad	Riesgo	Nivel	Justificación
Se aceptan tiempos negativos.	A06	3	2	6	Alto	Un juez puede guardar tiempos inválidos y alterar los resultados de la carrera. Se comprobó en Swagger.
El login no limita intentos en el código	A07	3	2	6	Alto	Facilita intentar adivinar credenciales y actuar como juez si se acierta la contraseña. No se

revisado y el formulario acepta contraseñas débiles.						demostró robo de una cuenta real.
Un juez puede consultar registros de equipos ajenos.	A01	2	2	4	Medio	La ruta de estado entrega información sin comprobar la pertenencia del equipo. Parte de los resultados es pública, lo que limita el impacto de confidencialidad.
Entradas incorrectas provocan errores internos o muestran detalles técnicos.	A10	2	2	4	Medio	Revela información de implementación y responde incorrectamente ante datos inválidos.
Los registros de login no identifican la cuenta intentada ni su origen.	A09	2	2	4	Medio	Dificulta investigar intentos sospechosos. Sí se registraron la hora y la ruta.
Hay dependencias con avisos de seguridad y diferencias entre las versiones declaradas.	A03	3	1	3	Medio	Una dependencia vulnerable puede causar daños importantes, pero falta comprobar qué avisos son explotables en Server5K.
Si falta SECRET_KEY, puede utilizarse una clave pública de respaldo.	A04	3	1	3	Medio	Permitiría falsificar tokens si un despliegue utiliza esa clave. Esa condición no se confirmó en la configuración local actual.
El modo DEBUG=True muestra trazas y rutas internas.	A02	2	1	2	Bajo en laboratorio, condicionado	Se reprodujo en desarrollo, no se confirmó que esté expuesto en producción. Se relaciona con M-04.

- **Línea base del proyecto de seguridad consolidada, primer componente del portafolio que sustentará la defensa final de la semana 16.**

6. Preguntas de Control:

1. **La superficie de ataque de una aplicación web no se limita a sus páginas visibles: incluye formularios, parámetros, cabeceras y componentes de terceros. ¿Por qué un inventario exhaustivo de la superficie de ataque constituye el punto de partida de cualquier auditoría de seguridad, y qué riesgos implica omitir un componente del inventario?**

   El inventario permite conocer todos los puntos por los cuales una persona o servicio puede interactuar con la aplicación, no solo sus páginas visibles incluyendo formularios, parámetros, API, autenticación, WebSockets, panel administrativo, servicios internos y dependencias, a partir de esa información se define qué se probará, con qué permisos y qué evidencias se requieren, en el caso nuestro el Server5K, inventariar tanto los endpoints de equipos como el canal WebSocket evita limitar la revisión a la interfaz pública, si se omite una ruta o componente, podrían permanecer sin revisar controles de acceso, exposición de tokens, entradas manipulables o configuraciones peligrosas, lo que genera una conclusión de seguridad incompleta.

2. **Al clasificar los activos con la tríada CIA, un mismo activo puede exigir controles distintos para la confidencialidad, la integridad y la disponibilidad. ¿Cómo determinaría usted qué pilar priorizar cuando los recursos son limitados? Justifique su respuesta con un ejemplo de su propio proyecto.**

   La prioridad debe establecerse según el valor del activo, el daño de una vulneración y la probabilidad de que ocurra, considerando además las obligaciones y operaciones del sistema, en el Server5K se propone priorizar la integridad de los registros de tiempos y resultados, porque una modificación no autorizada alteraría la clasificación de los equipos y reduciría la confianza en la competencia, por eso, los primeros controles recomendados son autorizaciones por juez y equipo, validaciones en el servidor y un registro auditable de modificaciones. La disponibilidad sigue siendo importante durante una carrera, por lo que deben mantenerse respaldos y mecanismos de recuperación; la confidencialidad se refuerza en datos de cuentas y credenciales. Esta priorización deberá ajustarse si las pruebas muestran mayor exposición en otro activo.

3. **Los mensajes de error detallados facilitan la depuración durante el desarrollo, pero constituyen una fuga de información en producción. ¿Qué prácticas permiten conciliar ambas necesidades, y qué información concreta no debería revelar jamás un mensaje de error dirigido al usuario final?**

   Para conciliar la necesidad de depurar errores durante el desarrollo y proteger la información en producción, es importante implementar un manejo seguro de errores. Durante el desarrollo, se pueden utilizar mensajes detallados y registros técnicos que faciliten la identificación y corrección de fallos. Sin embargo, en producción se debe desactivar el modo de depuración, mostrar mensajes de error genéricos y almacenar los detalles técnicos en registros seguros, accesibles únicamente para el personal autorizado. Además, se recomienda implementar un manejo centralizado de excepciones y utilizar códigos de referencia para identificar los errores. Los mensajes dirigidos al usuario final nunca deben revelar información sensible, como contraseñas, tokens de autenticación, consultas SQL, estructuras de bases de datos, rutas internas del servidor, configuraciones del sistema o datos personales. De esta manera, se facilita la detección y solución de problemas sin comprometer la seguridad del sistema ni proporcionar información que pueda ser aprovechada por posibles atacantes.

4. **El OWASP Top 10 es un documento de concientización, no un estándar de verificación. ¿Qué relación existe entre sus diez categorías de riesgo y las vulnerabilidades concretas observadas en su aplicación, y por qué la priorización de hallazgos debe considerar el impacto y la probabilidad, y no únicamente la severidad técnica?**

OWASP Top 10:2025 permite identificar los principales riesgos de seguridad en aplicaciones web, pero es necesario realizar pruebas para comprobar si existen en el sistema. En el caso del Server5K se analizarán problemas como el acceso no autorizado a registros, configuraciones inseguras, dependencias vulnerables, exposición de tokens y mensajes de error que revelen información interna. Para evaluar estos riesgos, se considera la probabilidad de que ocurran y el impacto que podrían tener en los datos, usuarios y funcionamiento del sistema. Con estos resultados, se seleccionarán los tres riesgos más importantes que requieren atención prioritaria.

7. Conclusiones

- La selección de Server5K permitió definir un caso de auditoría con interfaz web, API REST, WebSockets y servicios de persistencia. Su ejecución local documentada representa un punto de partida válido; sin embargo, el inventario deberá completarse con rutas, métodos, permisos y capturas verificables.
- La identificación de diez activos proporciona una base suficiente para aplicar la tríada CIA y proponer controles en diferentes capas. La integridad de los registros de tiempos y resultados, junto con la confidencialidad de credenciales y tokens, merece especial atención dada la finalidad del sistema.
- La revisión inicial describe indicios de configuraciones de desarrollo, dependencias potencialmente vulnerables y posibles problemas de autorización y manejo de excepciones. Estos elementos requieren validación con pruebas reproducibles antes de afirmar una vulnerabilidad explotable en el despliegue final.

Rúbrica de Evaluación (10 puntos)

Criterio	Excelente (2.5)	Bueno (1.5)	Regular (0.5)	Deficiente (0)
1. Selección y alcance del proyecto (Fase 1)	La aplicación seleccionada cumple todos los requisitos, ejecuta localmente y el repositorio con la ficha y el inventario de superficie de ataque está completo y organizado.	La aplicación cumple los requisitos con observaciones menores; repositorio completo con detalles de forma pendientes.	Aplicación con requisitos parciales o inventario con menos de cinco elementos documentados.	Sin aplicación funcional o sin repositorio de evidencias.
2. Tríada CIA y matriz de activos (Fase 2)	Matriz con seis o más activos, clasificación CIA correcta por activo y controles coherentes con las amenazas identificadas.	Matriz completa con observaciones menores de clasificación o de controles.	Matriz con menos de cuatro activos o clasificación confusa entre los pilares.	Sin matriz de activos o sin clasificación CIA.
3. Gestión de errores (Fase 3)	Cinco o más fugas documentadas con evidencia, severidad clasificada y correcciones conforme a las buenas prácticas.	Tres o cuatro fugas documentadas o correcciones genéricas.	Menos de tres fugas documentadas o evidencias incompletas.	Sin informe de gestión de errores.
4. Mapeo OWASP Top 10 (Fase 4)	Lista de cotejo completa con conclusión fundamentada por categoria y matriz de priorización justificada.	Lista con vacíos menores de verificación o priorización poco justificada.	Lista parcial o sin evidencias de verificación.	Sin lista de cotejo ni matriz de priorización.
5. Documentación,	Evidencias claras y	Evidencias con	Evidencias	Evidencias

evidencias y ética	trazables en el repositorio; trabajo realizado exclusivamente en el entorno propio o controlado.	observaciones de formato; cumplimiento de las reglas éticas del laboratorio.	insuficientes o desordenadas.	ausentes o prácticas fuera del entorno autorizado.

Nota: La Actividad Práctico–Experimental podrá ser utilizada por el docente con carácter demostrativo. En tal caso, tanto la calificación como la entrega del Reporte Técnico por parte de los estudiantes pueden ser opcionales.

8. Bibliografía:

- Haz Lopez, L. V., et al. (2024). Hacking Ética: Teoría y Prácticas. CID. ISBN: 978-1-967344-23-9.
- Vega Briceño, E. (2021). Seguridad de la Información. Editorial Área de Innovación y Desarrollo. ISBN: 978-84-122093-6-5.
- Whitman, M. E., & Mattord, H. J. (2018). Management of Information Security (6th ed.). Cengage Learning.
- OWASP Foundation. (2025). OWASP Top 10. Recuperado de https://owasp.org/Top10/
- Howard, M., & LeBlanc, D. (2003). Writing Secure Code (2nd ed.). Microsoft Press.